Практическое задание по дисциплине «Программная инженерия на языке программирования C++»:

Задание: разработать систему мониторинга автомобиля, которая одновременно отображает телеметрию из реального датасета OBD-II (скорость, обороты, температура, топливо) и анализирует состояние водителя через веб-камеру (усталость, отвлечение). Всё должно выполняться – в одном окне в режиме реального времени, выполнено на C++.

Приборная панель	Водитель
`87 km/h  3200 RPM`	`[лицо]`
`TEMP  92°C`	`Глаза: открыты`
`FUEL  68%`	`Голова: прямо`
`STYLE: NORMAL 91%`	`Статус: НОРМА`

Задание 0 – Установка окружения

Установить необходимые инструменты ниже. После каждого пункта проверьте в терминале что всё работает.

1. Visual Studio Code

- скачайте и установите: https://code.visualstudio.com;
- установите расширения: `C/C++, CMake Tools, GitLens`.

2. Git

- скачайте и установите: https://git-scm.com;
- проверить командой: `git -version.`

3. CMake 3.20+

- скачайте: https://cmake.org/download (файл `.msi` для Windows);
- при установке выбери: **Add CMake to the system PATH**;
- проверьте: `cmake -version.`

4. Компилятор C++ (MinGW-w64 для Windows)

- скачайте: https://winlibs.com → последний релиз Win64 с POSIX threads;
- распакуйте в `C:\mingw64;`
- добавьте `C:\mingw64\bin` в системный PATH;
- проверьте: `g++ --version.`

5. Python 3.10+

- скачайте: https://python.org (отметь **Add to PATH**);
- проверьте: `python -version.`

6. OpenCV 4.x

- скачайте: https://github.com/opencv/opencv/releases → `opencv-4.x.x-windows.exe;`
- распакуйте в `C:\opencv;`
- добавьте в PATH: `C:\opencv\build\x64\mingw\bin.`

7. ONNX Runtime

- скачайте: https://github.com/microsoft/onnxruntime/releases → `onnxruntime-win-x64-1.18.0.zip;`
- распакуйте в `C:\onnxruntime;`

Проверка всего окружения – выполните в терминале:

```
g++ --version
```

```
cmake --version
git --version
python --version
```

Все четыре команды должны вывести версию без ошибок.

Отчётные материалы:

скриншот терминала с четырьмя успешными командами.

Задание 1 – Архитектура проекта, CMake, первый коммит

Цель: создать структуру проекта, настроить систему сборки, сделать первый коммит в Git.

Шаг 1.1 – Создайте репозиторий на GitHub

1. Зайдите на https://github.com, создайте новый репозиторий `real-car-adas-monitor`.
2. Настройте Git на ноутбуке:

```
git config --global user.name "Ваше имя"
git config --global user.email "Ваш email"
```

Шаг 1.2 – Создайте структуру папок

Создайте папку `real-car-adas-monitor`. Внутри создайте такую структуру вручную:

```
real-car-adas-monitor/
├── CMakeLists.txt          ← главный файл сборки
├── README.md               ← описание проекта
├── .gitignore              ← что не загружать в Git
│
├── src/                    ← весь исходный код
│   ├── main.cpp
│   ├── obd_parser.h        ← парсер OBD CSV данных
│   ├── obd_parser.cpp
│   ├── onnx_classifier.h   ← ИИ-классификатор стиля вождения
│   ├── onnx_classifier.cpp
│   ├── dashboard.h         ← приборная панель (OpenCV)
│   ├── dashboard.cpp
│   ├── dms_monitor.h       ← детекция состояния водителя
│   ├── dms_monitor.cpp
│   ├── dms_hud.h           ← HUD для камеры
│   └── dms_hud.cpp
│
├── tests/
│   └── test_obd_parser.cpp ← unit-тесты
│
├── models/                 ← ONNX модели (не загружаются в Git)
├── data/                   ← CSV файлы (не загружаются в Git)
├── output/                 ← результаты (скриншоты, видео)
└── docs/
    └── architecture.md     ← описание архитектуры
```

Шаг 1.3 – Создайте CMakeLists.txt

Изучите структуру CMakeLists.txt. Файл должен:

- указывать стандарт C++17;
- находить OpenCV через `find_package`;
- находить ONNX Runtime по пути `C:/onnxruntime`;
- объявлять исполняемый файл `RealCarMonitor` из всех .cpp файлов в src/;
- подключать Google Test через `FetchContent`;
- объявлять исполняемый файл `RunTests` для тестов.

Документация CMake: https://cmake.org/cmake/help/latest/

Шаг 1.4 – Создайте минимальный main.cpp

На этом этапе main.cpp должен просто выводить приветствие и версию. Убедитесь, что проект собирается без ошибок.

Команды сборки:

```
mkdir build
cd build
```

```
cmake .. -G "MinGW Makefiles"
cmake --build .
./RealCarMonitor.exe
```

Шаг 1.5 – Создайте .gitignore и первый коммит

В `.gitignore` добавьте:

```
build/
*.exe
*.dll
models/*.onnx
data/*.csv
output/
```

Затем:

```
git init
git add .
git commit -m "feat: initial project structure and CMake setup"
git remote add origin https://github.com/НИК/real-car-adas-monitor.git
git push -u origin main
```

Отчётные материалы:

- ссылка на GitHub репозиторий;
- скриншот: проект успешно собрался;
- файл `docs/architecture.md` с вашей схемой;
- первый коммит в истории.

Задание 2 – Парсер OBD данных + unit-тесты

Цель: написать класс `OBDParser`, который читает CSV файл с телеметрией автомобиля. Покрыть его unit-тестами через Google Test.

Шаг 2.1 – Скачайте датасет

Вариант А – реальный датасет с Kaggle (можно использовать любой другой ресурс с готовыми датасетами):

1. Зарегистрируйтесь на https://www.kaggle.com.
2. Скачайте API-ключ: профиль → Account → API → Create New Token → скачается `kaggle.json`.
3. Найдите датасет с OBD-II телеметрией. Ключевые слова по запросу: `OBD driving speed RPM telemetry`.
4. Датасет должен содержать столбцы: скорость, обороты двигателя, положение дроссельной заслонки, температура охлаждающей жидкости, уровень топлива
5. Положите CSV в папку `data/`

Вариант Б – сгенерировать данные (если датасет не найден): Напишите Python скрипт `generate_obd_data.py` который генерирует CSV с реалистичными значениями. Скрипт должен создать 5000 строк со столбцами: `speed_kmh`, `engine_rpm`, `throttle_pos`, `coolant_temp`, `fuel_level`, `intake_air_temp`, `label` (SLOW/NORMAL/AGGRESSIVE).

Шаг 2.2 – Спроектируйте класс OBDParser

Прежде чем писать код – ответь на вопросы (эти вопросы будут заданы во время защиты):

- Какие данные хранит одна запись? → Создайте структуру `OBDRecord`.
- Что умеет делать парсер? → Опишите методы в заголовочном файле
- Как обрабатывать ошибки? (файл не найден, неверный формат строки)
- Как конвертировать строку "SLOW"/"NORMAL"/"AGGRESSIVE" в число?

Напишите `obd_parser.h` с объявлением структуры и класса (только заголовок, без реализации).

Шаг 2.3 – Напишите реализацию obd_parser.cpp

Реализуйте все методы. Требования:

- первая строка CSV – заголовок, пропустить;
- каждая строка – одна запись;
- если строка содержит некорректные данные – пропустить с предупреждением в `std::cerr;`
- метод `load()` возвращает количество загруженных записей или -1 при ошибке;
- метод `getRecord(int index)` бросает `std::out_of_range` при неверном индексе.

Шаг 2.4 – Напишите unit-тесты

В файле `tests/test_obd_parser.cpp` Напишите тесты с помощью Google Test:

- тест: конвертация меток (SLOW → 0, NORMAL → 1, AGGRESSIVE → 2);
- тест: загрузка несуществующего файла возвращает -1;
- тест: `getRecord()` бросает исключение при неверном индексе;
- тест: парсинг корректного CSV файла (Создайте временный файл в тесте, прочитайте, удалите);
- тест: парсинг файла с некорректной строкой (строка пропускается, остальные читаются).

Запустите тесты:

```
cmake --build . && ./RunTests.exe
```

Все тесты должны пройти: `[ PASSED ] 5 tests.`

Шаг 2.5 – Обновите main.cpp

В main.cpp загрузите CSV, выведи первые 5 записей и статистику по классам (сколько SLOW, NORMAL, AGGRESSIVE).

```
git add .
git commit -m "feat: OBD parser with unit tests"
git push
```

Отчётные материалы:

- скриншот: вывод программы с данными из CSV;
- скриншот: все тесты прошли;
- коммит в репозитории.

Задание 3 – Обучение модели в Google Colab

Цель: обучить нейросеть классификации стиля вождения на Python, экспортировать в ONNX для последующего использования в C++.

Шаг 3.1 – Откройте Google Colab

Перейдите: https://colab.research.google.com → Создайте новый ноутбук.

Шаг 3.2 – Загрузите данные

Загрузите `obd_data.csv` в Colab (кнопка загрузки файлов в левой панели или через `files.upload()`).

Шаг 3.3 – Исследуйте датасет

Напишите код который:

- загружает CSV через `pandas;`
- выводит размер, первые 5 строк, статистику;
- строит столбчатую диаграмму распределения классов.

Шаг 3.4 – Предобработка

- разделить на признаки X и метки y;
- закодировать метки через `LabelEncoder` (SLOW=0, NORMAL=1, AGGRESSIVE=2);
- нормализовать признаки через `StandardScaler;`
- разбить на train/test (80/20);
- **обязательно сохраните параметры нормализации** (mean и std) – они понадобятся в C++.

Шаг 3.5 – Обучите нейросеть

Используйте PyTorch. Архитектура модели:

- вход: 6 признаков;
- два скрытых слоя с ReLU и Dropout;
- выход: 3 класса.

Обучите 60 эпох, постройте графики Loss и Accuracy.
Ожидаемая точность: **85-95%**.

Шаг 3.6 – Экспортируйте в ONNX

```
torch.onnx.export(model, dummy_input, "driver_classifier.onnx",
                  input_names=['features'],
                  output_names=['class_scores'],
                  opset_version=12)
```

Сохраните параметры нормализации в `normalization_params.json`:

```
{
  "mean": [...],
  "std":  [...],
  "classes": ["SLOW", "NORMAL", "AGGRESSIVE"]
}
```

Скачайте оба файла и положите в папку `models/`.

Отчётные материалы:

- ссылка на Colab ноутбук (Файл → Поделиться);
- скриншот графиков обучения;
- скриншот финальной точности;
- файлы `driver_classifier.onnx` и `normalization_params.json` в репозитории.

Задание 4 – ONNX классификатор на C++

Цель: написать класс `ONNXClassifier`, который загружает обученную модель и классифицирует записи телеметрии.

Шаг 4.1 – Изучите ONNX Runtime API

Документация: https://onnxruntime.ai/docs/get-started/with-cpp.html
Ключевые классы которые понадобятся:

- `Ort::Env` – среда выполнения;
- `Ort::Session` – сессия модели;
- `Ort::Value::CreateTensor` – создание входного тензора;
- `session.Run()` – запуск inference.

Шаг 4.2 – Спроектируйте класс ONNXClassifier

Создайте `onnx_classifier.h`. Класс должен:

- загружать ONNX модель и JSON с параметрами нормализации;
- принимать массив из 6 float (признаки телеметрии);
- нормализовать их (z-score: `(x - mean) / std`);
- возвращать структуру `ClassificationResult` с полями: `label` (0/1/2), `confidence` (0..1), `scores` (массив из 3 вероятностей).

Шаг 4.3 – Реализуйте ONNXClassifier

Важные детали:

- на Windows путь к модели нужно передавать как `std::wstring;`
- после `session.Run()` применить **Softmax** к выходным логитам чтобы получить вероятности;
- обработать случай когда модель не загружена – бросить `std::runtime_error.`

Шаг 4.4 – Напишите минимальный JSON парсер

Файл `normalization_params.json` нужно прочитать без внешних библиотек. Напишите функцию которая:

- читает весь файл в строку;
- находит ключ `"mean"` и извлекает массив чисел;
- находит ключ `"std"` и извлекает массив чисел.

Шаг 4.5 – Проверьте в main.cpp

Загрузите первые 20 записей из CSV, классифицируйте каждую, выведите таблицу: истинная метка – предсказание – уверенность. Подсчитайте точность.

```
git commit -m "feat: ONNX classifier module"
```

Отчётные материалы:

- скриншот таблицы с 20 классификациями;
- точность должна быть 80%+.

Задание 5 – Приборная панель Dashboard

Цель: написать класс `Dashboard`, который отрисовывает приборную панель автомобиля поверх изображения OpenCV.

Шаг 5.1 – Изучите OpenCV рисование

Документация: https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html
Функции которые понадобятся:

- `cv::circle` – круги (циферблаты);
- `cv::ellipse` – дуги (стрелки спидометра);
- `cv::rectangle` – прямоугольники (фон, полосы);
- `cv::putText` – текст;
- `cv::line` – линии;
- `cv::addWeighted` – полупрозрачное наложение.

Шаг 5.2 – СпроектируйтеDashboard

Панель занимает левую половину кадра (640×480 пикселей). Должна содержать:

- **спидометр** – круговой прибор, 0–140 км/ч. Цвет дуги: зелёный (<90), красный (>90);
- **тахометр** – круговой прибор, 0–6000 RPM. Красная зона после 4500;
- **температура охлаждающей жидкости** – горизонтальная полоса, 0–120°C;
- **уровень топлива** – горизонтальная полоса, 0–100%;
- **дроссельная заслонка** – горизонтальная полоса, 0–100%;
- **стиль вождения** – текст с цветом (зелёный=NORMAL, красный=AGGRESSIVE, голубой=SLOW);
- **предупреждения** – если температура > 100°C или топливо < 15%.

Шаг 5.3 – Реализуйте Dashboard

Разбейте на приватные методы:

- `drawGauge()` – один круговой прибор (переиспользуется для спидометра и тахометра);
- `drawLinearGauge()` – одна горизонтальная полоса (переиспользуется для всех полос);
- `draw()` – главный метод, собирает всё вместе.

Шаг 5.4 – Проверьте отдельно

Создайте тестовый main.cpp, который создаёт статичный кадр, рисует на нём панель с тестовыми данными и показывает через `cv::imshow`. Убедитесь, что всё выглядит корректно.

```
git commit -m "feat: dashboard module with gauges"
```

Отчётные материалы:

- скриншот приборной панели с тестовыми данными;
- все 4 прибора отображаются корректно.

Задание 6 – Driver Monitoring System (DMS)

Цель: написать класс `DMSMonitor`, который через веб-камеру определяет состояние водителя.

Шаг 6.1 – Скачайте модели детекции

Детектор лиц (DNN): скачайте два файла и положите в `models/`:

- `deploy.prototxt` – архитектура сети;
- `res10_300x300_ssd_iter_140000.caffemodel` – веса.

Ссылки:

`https://raw.githubusercontent.com/opencv/opencv/master/samples/dnn/face_detector/deploy.prototxt`

`https://github.com/opencv/opencv_3rdparty/raw/dnn_samples_face_detector_20180220_uint8/res10_300x300_ssd_iter_140000_uint8.pb`

Детектор глаз (Haar Cascade): Файл уже есть в OpenCV. Скопируйте:

`C:\opencv\build\etc\haarcascades\haarcascade_eye.xml → models/`

Шаг 6.2 – Спроектируйте DMSMonitor

Создайте структуру `DriverState`:

```
Face_detected    - лицо обнаружено (bool);
eyes_open        - глаза открыты (bool);
looking_forward  - смотрит прямо (bool);
eye_openness     - степень открытости 0.0-1.0 (float);
head_turn_deg    - угол поворота головы (float);
alert_drowsy     - алерт: усталость (bool);
alert_distracted - алерт: отвлечение (bool);
face_rect        - координаты лица на кадре (cv::Rect).
```

Класс `DMSMonitor` должен:

- кагружать обе модели;
- метод `analyze(frame)` – анализирует кадр, возвращает `DriverState;`
- хранить историю последних 15 кадров для определения устойчивого закрытия глаз;
- алерт усталости: глаза закрыты в 10 из 15 последних кадров.

Шаг 6.3 – Реализуйте методы

Разбейте на приватные методы:

- `detectFace()` – возвращает `cv::Rect` лучшего лица с уверенностью > 0.5;
- `estimateEyeOpenness()` – ищет глаза в верхней половине ROI лица;
- `estimateHeadTurn()` – смещение центра лица от центра кадра → угол в градусах.

Шаг 6.4 – Создайте DMSHUD

Класс для отображения DMS данных на правой половине кадра:

- вставьте изображение с камеры в правую половину;
- нарисуйте рамку лица (угловые линии, не полный прямоугольник);
- нарисуйте индикаторы состояния (цветные точки + текст);
- алерт усталости: оранжевая рамка + плашка в центре;
- алерт отвлечения: красная полоса снизу.

Шаг 6.5 – Проверьте отдельно

Запустите только DMS с камерой. Проверьте:

- ваше лицо определяется → зелёная рамка;
- закройте глаза на 2 секунды → появляется DROWSINESS ALERT;
- повернитесь в сторону → появляется DISTRACTION.

```
git commit -m "feat: driver monitoring system with alerts"
```

Отчётные материалы:

- 3 скриншота: норма, усталость, отвлечение.

Задание 7 – Интеграция: многопоточная финальная система

Цель: объединить все модули в одно приложение с многопоточной архитектурой.

Шаг 7.1 – Спроектируйте многопоточную архитектуру

Система работает в двух потоках:

```
Поток 1 (OBD): читает CSV, классифицирует → пишет в SharedState
Поток 2 (Main): читает камеру, анализирует водителя,
                читает SharedState, рисует кадр 1280×480
```

Шаг 7.2 – Создайте SharedState

Структура, которая хранит:

- текущую OBD запись;
- результат классификации;
- счётчик алертов;
- флаг `running` типа `std::atomic<bool>`;
- `std::mutex` для защиты данных.

Шаг 7.3 – Напишите OBD поток

Функция запускается в `std::thread`. Каждые 100 мс (10 Hz) и она:

1. Берёт следующую OBD запись.
2. Классифицирует стиль вождения.
3. Захватывает mutex и обновляет SharedState.
4. Проверяет флаг `running` – если false, завершается.

Шаг 7.4 – Напишите главный цикл

Главный поток:

1. Читает кадр с веб-камеры.
2. Анализирует водителя через `DMSMonitor::analyze()`.
3. Захватывает mutex и читает текущие OBD данные.
4. Создаёт кадр 1280×480: левая половина – Dashboard, правая – DMS.
5. Записывает в `output/result_situation2.mp4`.
6. Логирует алерты в `output/dms_alerts.log`.

Управление: Q – выход, SPACE – пауза, S – скриншот.

Шаг 7.5 – Добавьте итоговую статистику

После выхода выведите в консоль:

- время работы системы;
- количество обработанных OBD записей;
- количество алертов всего;
- из них: усталость / отвлечение / агрессивное вождение.

Шаг 7.6 – Финальные тесты

Добавьте тесты для `DMSMonitor`:

- тест: по умолчанию модуль не загружен;
- тест: `analyze()` на пустом кадре не падает, возвращает `face_detected = false;`
- тест: загрузка моделей (если файлы есть).

Запустите все тесты – должны пройти успешно.

Шаг 7.7 – Документация Doxygen

1. Добавьте Doxygen-комментарии к каждому классу и публичному методу (формат `/** ... */`).
2. Создайте файл `Doxyfile` в корне проекта.
3. Запустите: `doxygen Doxyfile`.

4. Откройте `docs/doxygen/html/index.html` в браузере.

Шаг 7.8 – Обнови README.md

README должен содержать:

- описание что делает система;
- таблица стека технологий;
- инструкция: как собрать (команды cmake);
- инструкция: как запустить;
- скриншот работающей системы;
- таблица результатов (FPS, точность, количество алертов).

Финальный коммит:

```
git add .
git commit -m "feat: complete ADAS system - multithreading, tests, docs"
git push
```

Отчётные материалы:

- ссылка на GitHub репозиторий (финальная версия);
- `output/result_situation2.mp4` – запись работающей системы;
- 3 скриншота (нажми S): норма, усталость, агрессивное вождение;
- `output/dms_alerts.log`;
- скриншот: все тесты прошли успешно;
- скриншот: документация Doxygen в браузере.